covariance. In other words, we are doing nothing more than creating a multivariate Gaussian distribution with a mean vector and a covariance matrix computed from the `mean_module` and `covar_module` attributes of our model class.

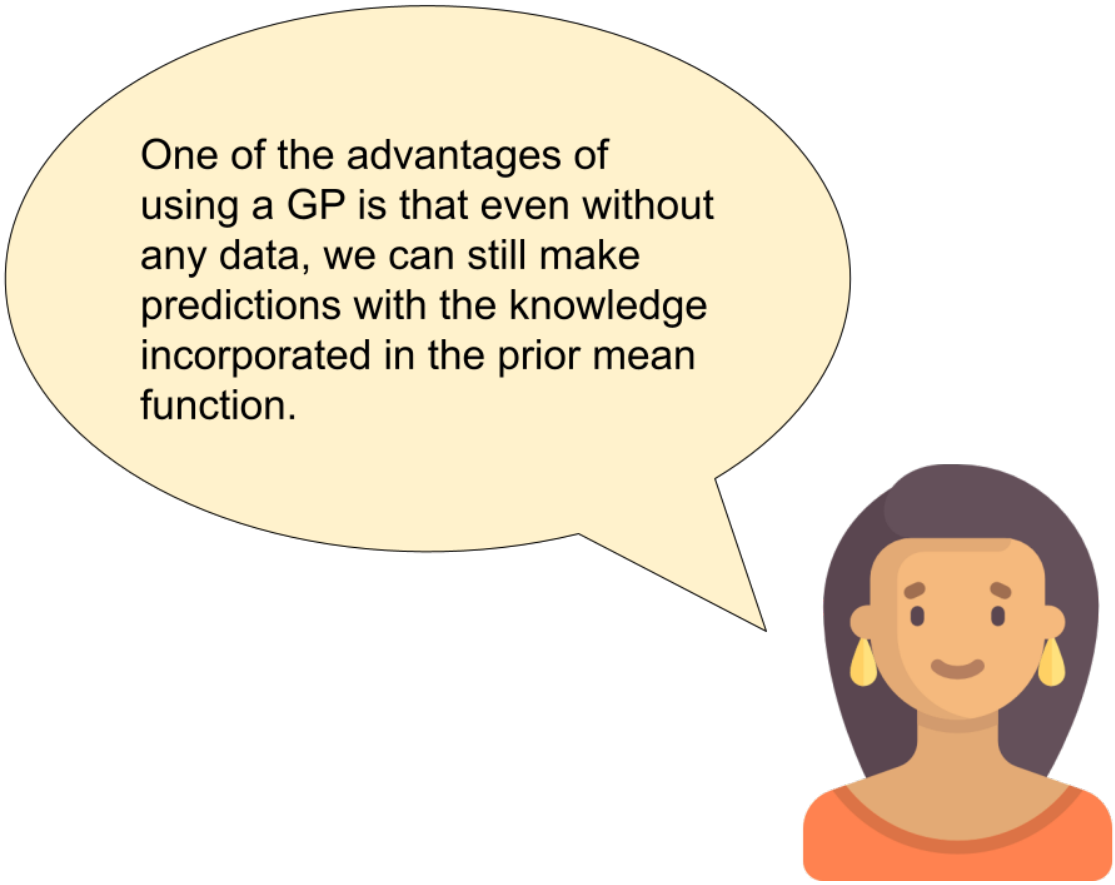
And that is all there is to it! It's quite surprising how easy it is to implement a GP model with GPyTorch. The biggest takeaway for us is that we need to implement a mean function and a covariance function in the `__init__()` method, and in the `forward()` method when we need to make a prediction, we simply call these two functions on the input that is passed in.

2.4.3 Making predictions with a Gaussian process

With the `BaseGPModel` class in hand, we are ready to make predictions with a GP! Recall that in the `__init__()` method, we need to pass in a likelihood function `likelihood` in addition to our training data. In many regression tasks, a `gpytorch.likelihoods.GaussianLikelihood` object suffices. We create this object like so:

```
likelihood = gpytorch.likelihoods.GaussianLikelihood()
```

Now, we can initialize our `BaseGPModel` object. But before we initialize it with our three-entry training data, we would like to first make predictions with the *prior* GP.



©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>